



Python

Introduction to Computer Programming

Quick overview

SCI-SHU 11

Fall 2026

Fang Yuan
fy12@nyu.edu

- Instructor
- Course Organization
- Grading
- Course content

Instructor

Fang Yuan

- fy12@nyu.edu

- Office S721

- Office hours:

Tue. 1:30pm – 3:00pm (Zoom: 91297633964)

Wed. 10:00pm – 11:30pm (Zoom: 98292134584)

+ extra by appointment

Recitation Instructors

Yuntong Hu

- yuntong.hu@emory.edu
- Session 8
- Office: TBD
- Office hours:
Thursday 2pm-3pm

Yilin Sun

- yilin.sun@sjtu.edu.cn
- Session 5&9
- Office: TBD
- Office hours:
Friday 3pm-5pm

Sihang (Nagi) Zhao

- sz3707@nyu.edu
- Session 7
- Office: TBD
- Office hours:
Tuesday 3pm-4pm

Text Book

Starting Out With Python, sixth edition Tony Gaddis, Pearson

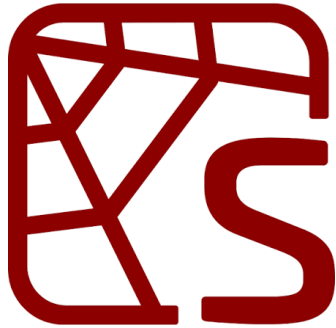
starting out with >>> **PYTHON**[®]
SIXTH EDITION



TONY GADDIS

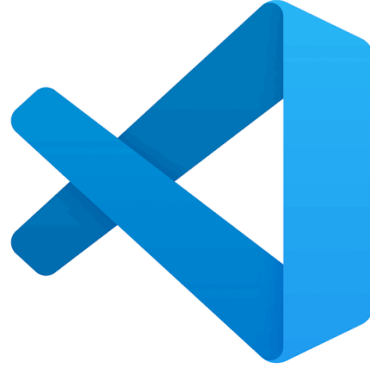
Software

Spyder



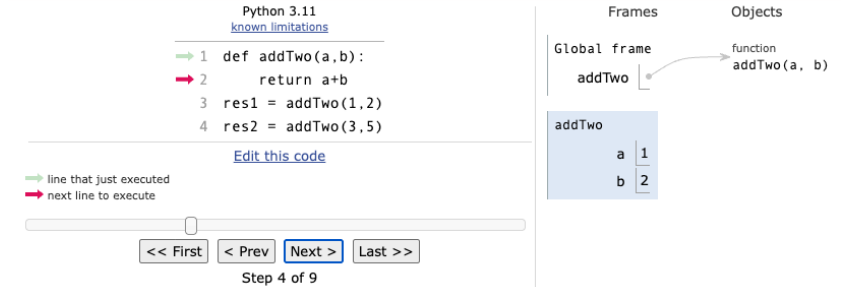
(recommend)
Beginner-friendly

VScode



one of the most widely used
code editors among software
developers and engineers

Python tutor



You can codes on the wed site, and
can see what happens step-by-step as
the computer runs it. It visualizes
variables, objects, pointers, data
structures, and stack frames.

Course Schedule

Date	Content	Quiz
8.31 week 1	Introduction	
9.7 week 2	Input, Processing, and Output	
9.14 week 3	Decision Structure	
9.21 week 4	Repetition Structure 1	Quiz 1
9.28 week 5	Repetition Structure 2	
10.5 week 6	National Day Holiday	
10.12 week 7	Midterm review	Quiz 2
10.19 week 8	Midterm	
10.26 week 9	Functions	
11.2 week 10	List and Tuples	
11. 9 week 11	Files and Exceptions	Quiz 3
11.16 week 12	Dictionary and Set	
11.23 week13	OOP	
11.30 week 14	Final Review	Quiz 4
12.7 week 15	Final Exam Mock	
12.14	Final	

Grading

Grading Items

■ Final Exam	35%
■ Midterm Exam	30%
■ Quizzes(4X 5%)	20%
■ Homework	10%
■ Attendance	5%

Grading

Homework assignments

- 8 Homework assignments Questions posted at BrightSpace, your solutions uploaded on Gradescope
- Deadline: Sunday Nights for the week (Each assignment should not take longer than 1 hour)

Attendance and Participation

- Attendance is mandatory and will be taken during class
- You start with a 5% grade, missed class: -0.5%
- Class participation is important

Grading

Grading scale

Grade	Minimum %
A	95
A-	90
B+	87
B	83
B-	80
C+	77
C	73
C-	70
D+	67
D	63
F	0

How to get top grades?

- Attend class (participation)
- Do reading assignments
- Work on question sets at home
- Use pen & paper
- Submit homework assignments on time
- Ask questions
- ...

Course Content

- Overview of programming concepts
- Integrated Development Environment (IDE)
- Data types and operators
- Conditional statements
- Loops
- Functions
- Files and Exceptions
- Lists and Tuples
- Dictionaries and Sets
- OOP(Object-Oriented Programming)

NYU
上海



SHANGHAI
纽约大学

Python

Introduction to Computer Programming

Introduction

SCI-SHU 11

Fall 2026

Fang Yuan
fy12@nyu.edu

Computers



- send emails
- write reports
- browse the web
- play games
-

Smartphones are computer systems

Hardware and Software

- Physical devices that a computer is made of are referred to **hardware**.
- Programs are referred to **software**.
- A computer by itself does not perform any task, but it can run programs

Typical hardware system

- Central Processing Unit (CPU)
- Random-Access Memory (RAM)
- Storage devices (Hard drives, SSD, . . .)
- Input devices
- Output devices

Computers



- send emails
- write reports
- browse the web
- play games
-

Smartphones are computer systems

Smartphone example

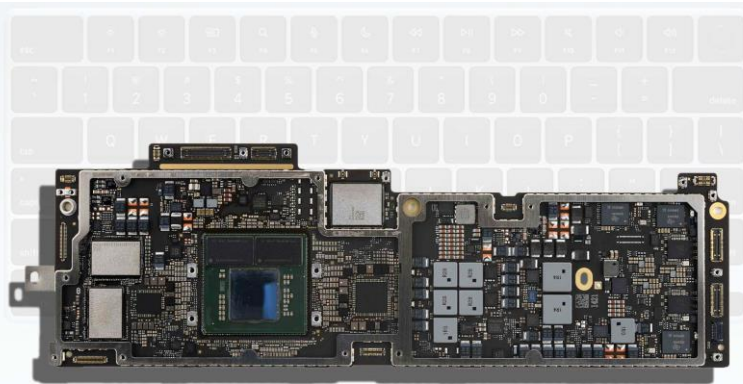
- CPU: Apple 19, Snapdragon 8 Elite Gen 5, . . .
- RAM: 8 – 64 GB
- Storage: typically 128 GB – 1 TB
- Input devices: Buttons, touchscreen, microphone . . .
- Output devices: Screen, vibrations, speakers, . . .

Hardware

CPU

Most important part in a computer

- Actually runs programs
- also known as **microprocessors**
- performs operations
(addition, subtraction,
multiplication, . . .
and more complex instructions)



Apple's M5 MacBook Air Lineup

RAM

Computer's work area

- where the computer stores a running program and the data associated to this program
- Volatile memory → needs power to store data
- when power is off, the contents of RAM are lost



Storage

Can hold data for long periods of time, even without power

- Internal or external for user data
- Hard drive, USB drives,



Hardware

Input Devices

Input devices collect data from people and from other devices. Data can be:

- Keyboards (text, numerical values)
- Mics (sound)
- Webcams (video/sound)
-



Output Devices

- Presents\produces data
Monitors
Printers

.....



Software(Program)

- Computer hardware is useless without software
- Software is the set of instructions and associated data that direct the computer to do a task

2 categories of software

- System software
 - Operating systems (Windows, Linux, MacOS, Android, iOS, . . .)
 - Utility programs (antivirus, . . .)
- Software development tools (IDLE, . . .)
 - Application software
 - programs you usually launch on your computer/smartphone
example: Chrome, Power-point, Wechat, . . .

How Computers Store Data?

Numbers

integers (int):

(+): 0, 1, 100

(-): -1, -2, -100

decimal numbers (float):

(+): 0.1, 12.333

(-): -0.1, -10.0, -11.2222

with decimal points

Text

Characters:

A, z, ICP, python...

Punctuations:

., :, -, #, ?, !

Chinses:

上海, 你好

Others types of data

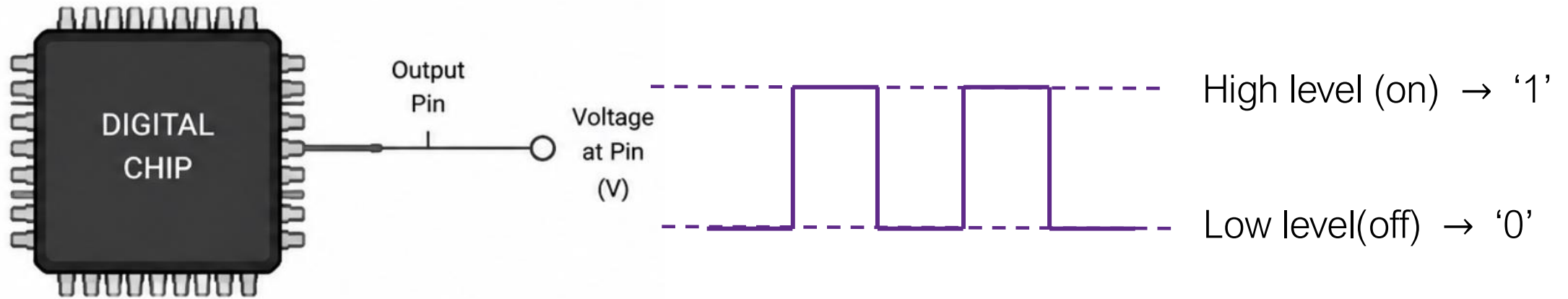
Image

Sound

Video

How Computers Store Data?

All data that is stored in a computer is converted to sequences of **0s and 1s**.



- Computers can only handle **binary** information.
- A computer's memory is divided into tiny storage locations known as **bytes (B)**.
- Each byte is divided into eight smaller storage locations known as **bits**.
two states for a bit: 0 (off) and 1 (on)

Storing integers

A byte is a group of 8 bits, i.e. 1 byte (or 1B) = 8 bits (or 8b)

Bit Position → 7 6 5 4 3 2 1 0

1	0	0	1	0	0	1	1
---	---	---	---	---	---	---	---

1 Byte (1 B)

0b10010011=147

a binary preceded by '0b'

- Position values:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

- Integer value = $\sum (\text{digit} \times \text{Position values})$

$$1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 147$$

147 is a base-10 number, also called a decimal number.

Storing integers

Groups of bits

- A single bit can only take 2 different values: 0 or 1
- A group of 2 bits can take 4 different values: 00, 01, 10 or 11
- A group of 3 bits can take 8 different values: 000, 001, 010, 011, 100...
- ...
- A group of n bits can take 2^n different values.

It will represent integers between 0 and $2^n - 1$ (all bits are 0 to all bits are 1)

e.x. 1byte can represent $2^8 = 256$ different values (0~255)

A larger number?

two bytes (0~ $2^{16} - 1$, 0~65535), three bytes,

You try it!

How many different values can be represented by:

- 1 bit (1 b)?
- 2 bits (2 b)?
- 8 bits (8 b)?
- 1 byte (1 B)?
- 2 bytes (2 B)?
- 4 bytes (4 B)?

Storing integers

To store a decimal number (base 10) in a computer, we need to convert it to binary.

But how do we do that?

Storing integers - Decimal to Binary

Way 1: Sum of power of 2 (position values)

$$89 = 64 + \dots$$

$$= 64 + 25$$

$$= 64 + 16 + \dots$$

$$= 64 + 16 + 9$$

$$= 64 + 16 + 8 + \dots$$

$$= 64 + 16 + 8 + 1$$

$$= 2^6 + 2^4 + 2^3 + 2^0$$

$$= 0 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Similar to decimal numbers (e.x. 106):

$$1 \quad 0 \quad 6$$

$$10^2 \quad 10^1 \quad 10^0$$

$$106 = 1 \times 10^2 + 0 \times 10^1 + 6 \times 10^0$$

$$89 = 0b01011001$$

Storing integers - Decimal to Binary

Way 2: Successive divisions by 2 until the quotient is 0

2	89	
2	44	 1
2	22	 0
2	11	 0
2	5	 1
2	2	 1
2	1	 0
0		 1
Quotient	Remainder	

Read the remainders from bottom to top:

$89 = 0b1011001$ 7 bits

↓ extend it to 1 byte

$89 = 0b01011001$ 8 bits

How can we extract each digit from 89?

10	89	
10	8	 9
0		 8

Storing integers - Binary to Decimal

$$\text{Integer value} = \sum (\text{digit} \times \text{Position values})$$

Binary: 0b10010011

Decimal:

$$1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 147$$

$$0b10010011 = 147$$

Hexadecimal

Can you read it?

0b11111110110111001011101010011000

Long binary numbers are difficult for **humans** to read.

The key observation is: $16 = 2^4$. One hexadecimal digit represents four binary bits.

Hexadecimal is a base-16 number system (preceded by **0x**):

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
										10	11	12	13	14	15

Binary: 0b 0011 0110 1101 1100

Hexadecimal: 0x 36DC

Decimal $3 \times 16^3 + 6 \times 16^2 + 13 \times 16^1 + 12 \times 16^0$

Where Do We See Hexadecimal?

Memory addresses: 0x7FF0A120 ;

IPv6 address

Colors: #FF0000 Red ; #00FF00 Green ; #0000FF Blue

You Try it!

Convert to decimal

- 0b000000010
- 0b00100100
- 0b11111111
- 0x5
- 0xA4

Convert to binary

- 12
- 164
- 300

Try `bin()` and `hex()` functions in interactive mode to check your answer:

```
>>> bin(12) # return the binary of 12
```

```
>>> hex(164) # return the hexadecimal of 164
```

```
>>> 0b00100100 # return the decimal of 0b00100100
```

Storing numbers

Negative numbers

- A slight different strategy is usually used to store negative integers
- It is two's complement.
- not covered in this course

Decimal point

- two strategies exist to encode decimal point numbers:
- Fixed-point → binary point stays fixed
- Floating-point → binary point can “float” through the exponent
- no covered in this course

Storing characters

- Any piece of data that is stored in a computer's memory must be stored as a binary number.
- That includes characters, such as letters and punctuation marks.



Convert a character to a numeric code.

One coding schemes is: American Standard Code for Information Interchange (ASCII)

- Table of correspondences between numbers (on 7 bits) and characters → 128 characters
- Limitations
 - Chinese characters?
 - Accented letters?
 - Mathematical symbols?
 - Currency symbols?
 - ...

the Unicode character

ASCII Code

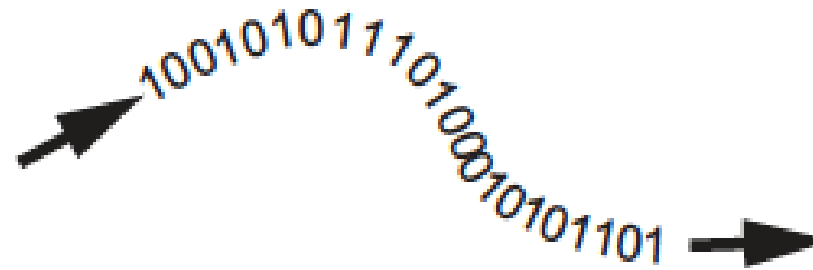
Hex Symbol	Hex Symbol	Hex Symbol	Hex Symbol	Hex Symbol	Hex Symbol
20 (space)	30 0	40 @	50 P	60 `	70 p
21 !	31 1	41 A	51 Q	61 a	71 q
22 "	32 2	42 B	52 R	62 b	72 r
23 #	33 3	43 C	53 S	63 c	73 s
24 \$	34 4	44 D	54 T	64 d	74 t
25 %	35 5	45 E	55 U	65 e	75 u
26 &	36 6	46 F	56 V	66 f	76 v
27 '	37 7	47 G	57 W	67 g	77 w
28 (	38 8	48 H	58 X	68 h	78 x
29)	39 9	49 I	59 Y	69 i	79 y
2A *	3A :	4A J	5A Z	6A j	7A z
2B +	3B ;	4B K	5B [	6B k	7B {
2C ,	3C <	4C L	5C \	6C l	7C
2D -	3D =	4D M	5D]	6D m	7D }
2E .	3E >	4E N	5E ^	6E n	7E ~
2F /	3F ?	4F O	5F _	6F o	7F

Storing other type of data

Convert any data into **binary** numbers. (Digital data)

Picture
Sound
Video

.....



Programming

Programming is about writing 'recipes' to generate facts

Example

Let the computer get: the square root of a number x is y such that $y*y=x$

The recipe (**Algorithm**):

Start with a guess, g

- 1) If $g*g$ is close enough to x , stop and say g is the answer
- 2) Otherwise make a new guess by averaging g and x/g
- 3) Using the new guess, repeat process until close enough

Let's try it for $x = 16$
and an initial guess of 3

g	$g*g$	x/g	$(g+x/g)/2$
3	9	$16/3$	4.17
4.17	17.36	3.837	4.0035
4.0035	16.027		

Programming

We have an algorithm:

- 1) Sequence of simple steps
- 2) Flow of control process that specifies when each step is executed
- 3) A means of determining when to stop

Key insight:

Programs are no different from other kinds of data. They are stored in memories, and will be encoded into binary numbers (it is called instruction set), and then CPU perform those instructions.

How a Program Works

Program in Python

```
x = 5
y = 3
z = x + y
```

Interpreted (compiled) into
Assembly Language

```
MOV R1, 5
MOV R2, 3
ADD R1, R2
MOV z, R1
```

Machine Language

```
0001 0001 00000101
0001 0010 00000011
0010 0001 00100010
0001 0001 00000000
```

The program is copied
from secondary storage
to main memory.



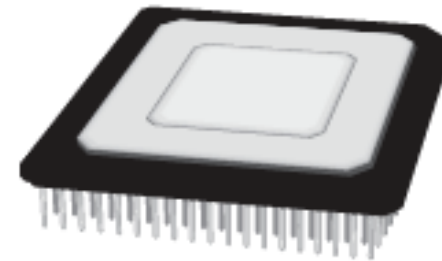
Disk drive

10100001 10111000 10011110



Main memory
(RAM)

The CPU executes
the program in
main memory.



CPU

CPU

How a Program Works

Program in Python

```
x = 5  
y = 3  
z = x + y
```

Java, JavaScript
C++, C
Python, C#
.....

High level languages

Interpreted (compiled) into Assembly Language

```
MOV R1, 5  
MOV R2, 3  
ADD R1, R2  
MOV z, R1
```

A human-readable
representation of
machine language

Low-level languages

Machine Language

```
0001 0001 00000101  
0001 0010 00000011  
0010 0001 00100010  
0001 0001 00000000
```

The CPU can directly
understand and execute.

Programming

Machine Language

- Example: 0001 00000101
- Written in binary (0s and 1s)
- Directly understood and executed by the CPU
- Almost unreadable to humans 🤪

Assembly Language

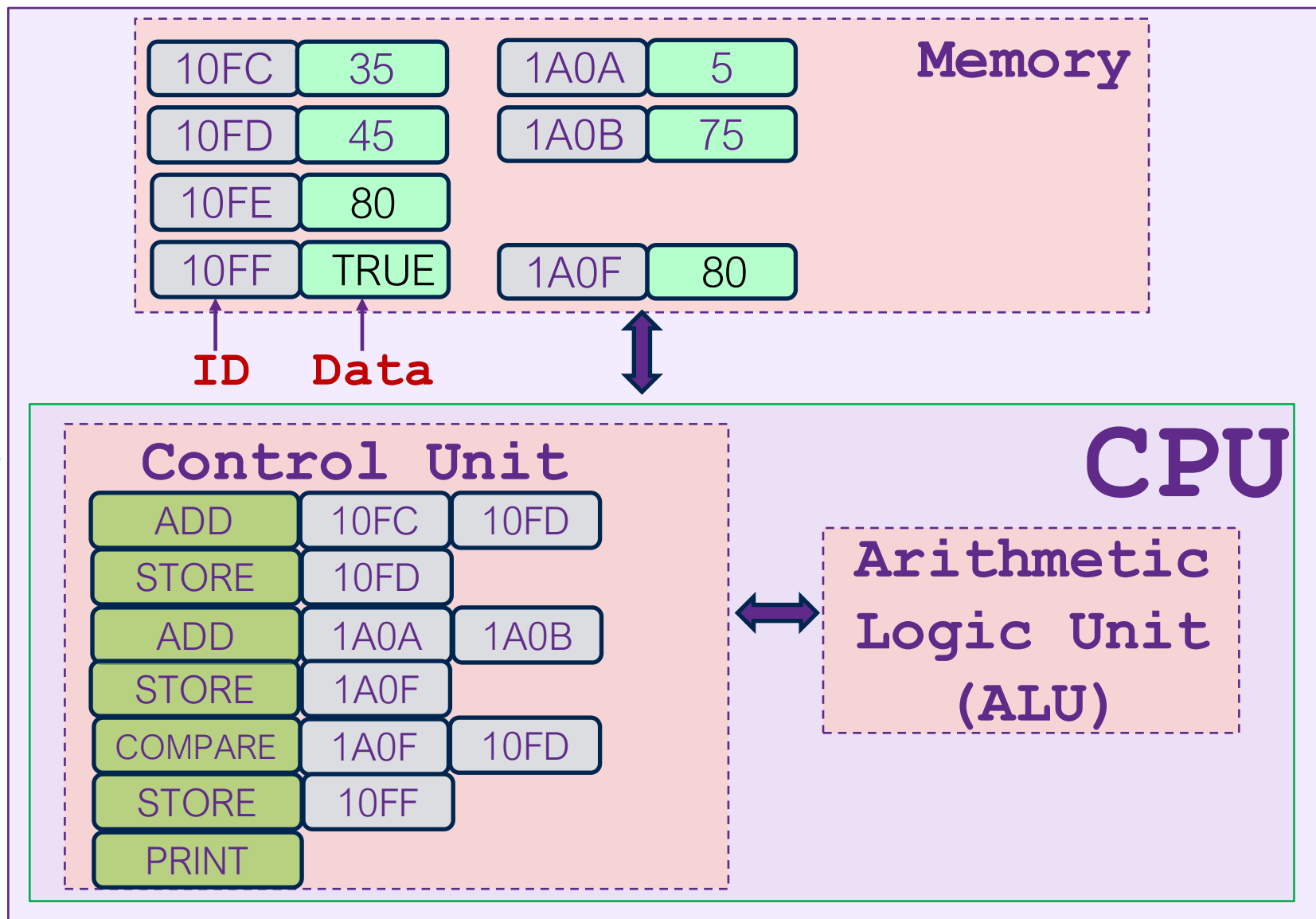
- Example: MOV R1, 5
- More readable than machine language
- Still low-level and hardware-specific
- Writing a large program in assembly would be a disaster 🤪 🌟

High-Level Languages

- Examples: Python, C++, Java
- Much easier for humans to read, write, and understand
- More similar to human language and mathematical notation
- Must be translated into machine code before the CPU can execute it.



Input →



→ Output
TRUE

Memory stores programs and data.

The Control Unit fetches and decodes instructions.

The ALU performs arithmetic and logical operations.

So many programming language

Popularity (based on TIOBE index on August 2026):

1. Python

2. C

3. C++

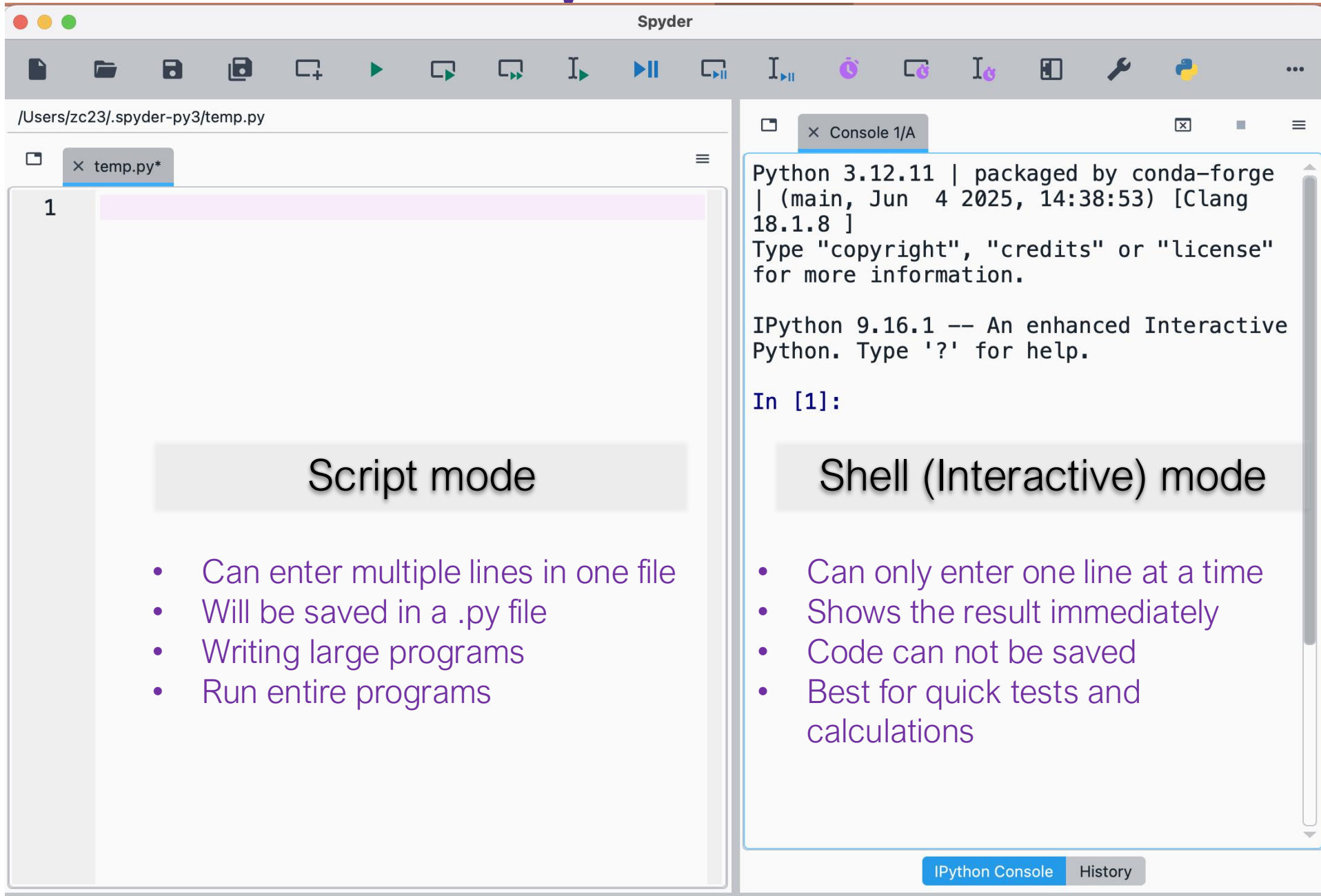
4. Java

5. C#

...

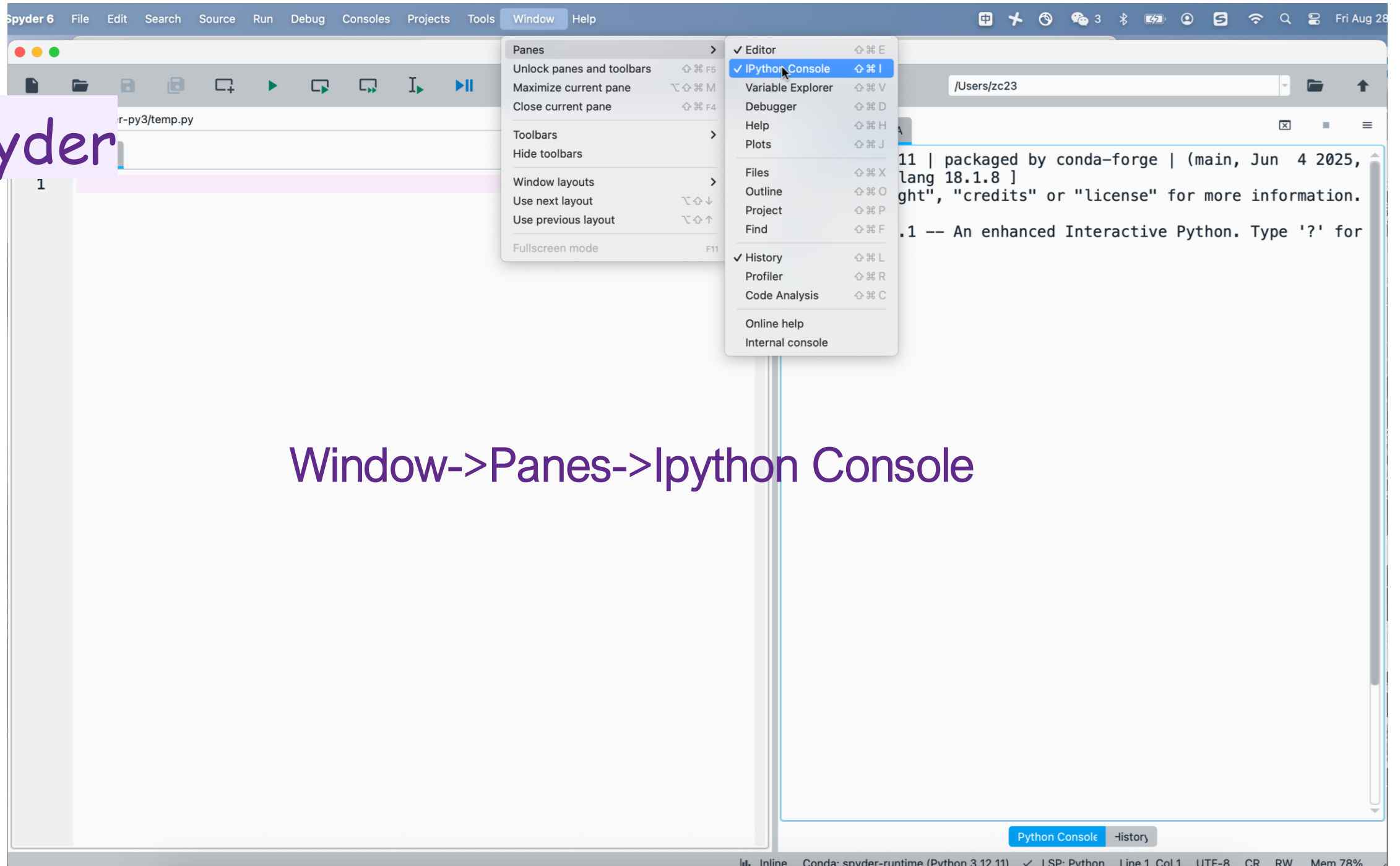
<http://www.tiobe.com/tiobe-index/>

Python IDE



Spyder

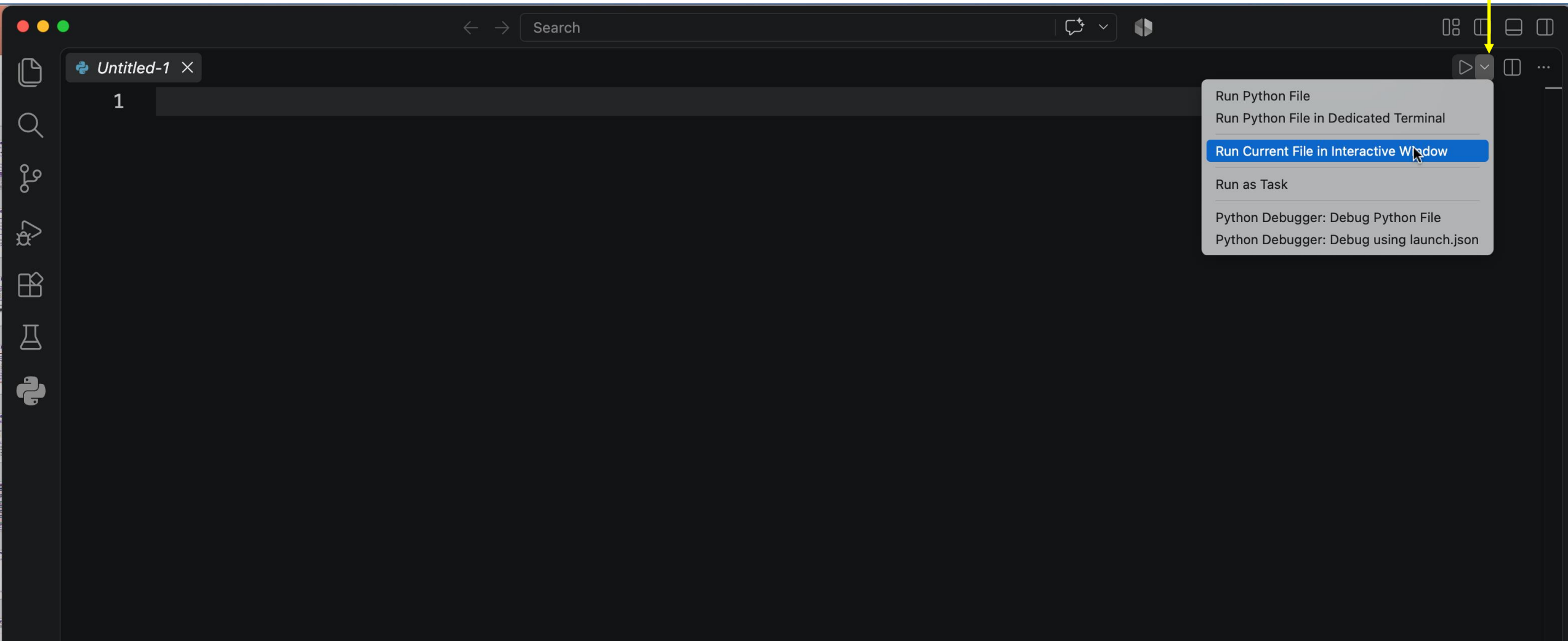
Window->Panels->Ipython Console



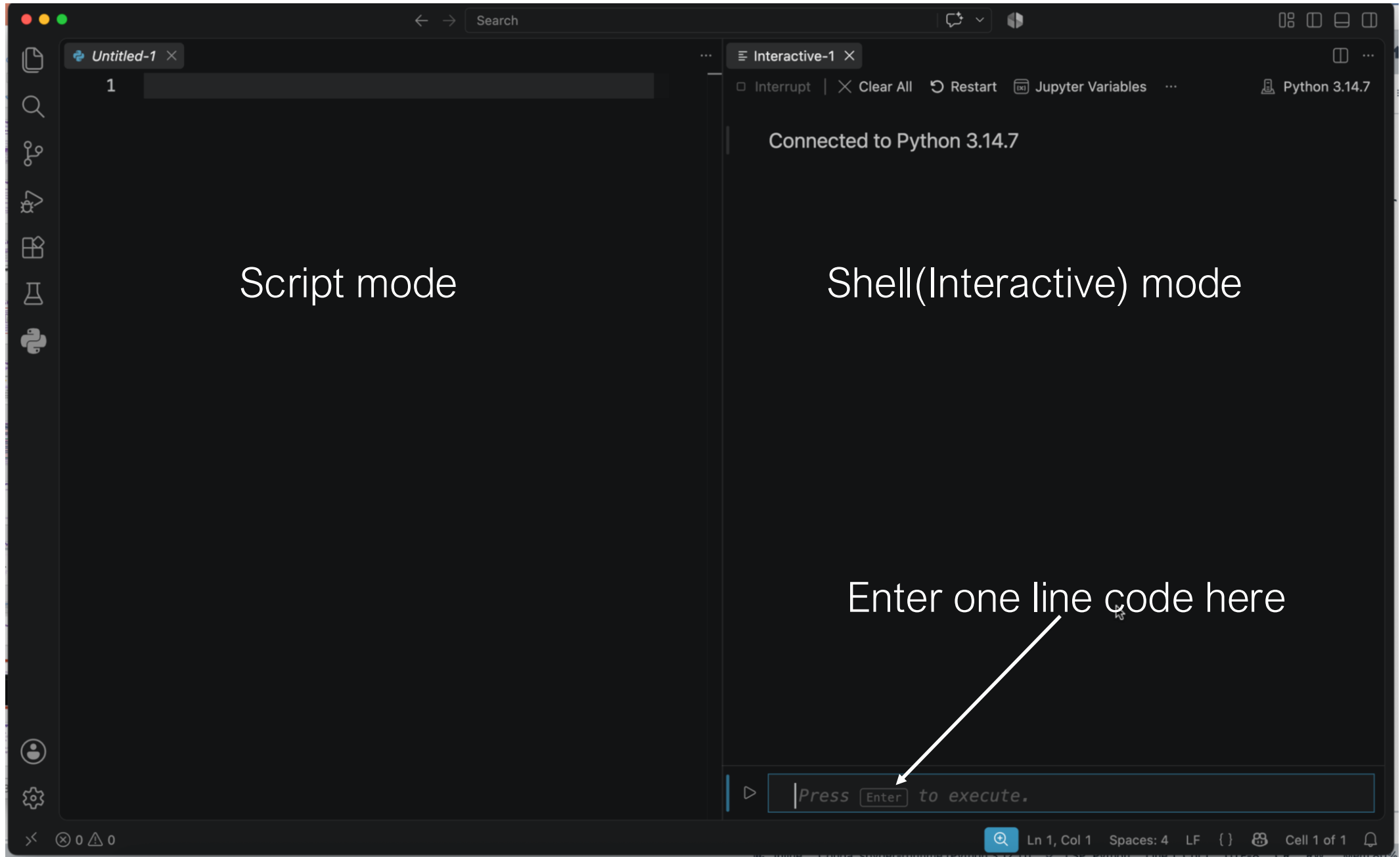
VScode

Click here

Create a new file->Choose “Run current file in interactive window”



VScode



Readings & Assignments

- This week readings
 - Read chapter 1 and 2 in the textbook
- This week recitation
 - Read recitation 01
 - Install IDLE Python 3.14 , Spyder (recommended) or Vscode.
- No Homework assignment this week.